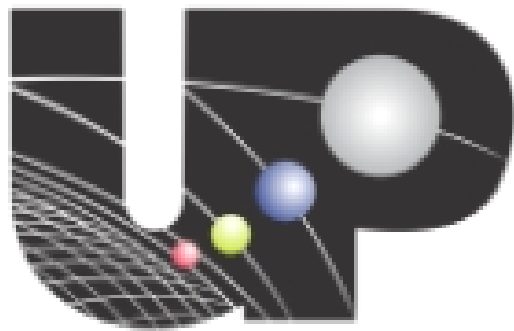




Universidad Politécnica de Chiapas

Evaluación Práctica Unidad 1

AWOS

Profesor:
Viviana Lopez Rojo

Calderón Trejo María Regina - 243723

5B

06 Febrero 2026

Índice

Justificación.....	3
1. Escenario.....	3
2. Stack.....	3
3. Estructura del Trabajo.....	3
4. Flujo de inicialización.....	3
5. BD.....	4
Shema.sql.....	4
Seed.sql.....	4
Indexes.sql.....	4
Reports_vw.sql.....	5
Roles.sql.....	6
6. Conexión de la BD al Frontend.....	6
7. Docker-compose.....	7
8. Variables de entorno.....	7
.env.....	7
.env.local.....	7
9. Interfaz.....	8
- Dashboard.....	8
- Rendimiento por curso.....	8
- Carga Docente.....	9
- Alumnos en Riesgo.....	9
- Asistencia por Grupo.....	10
- Ranking de Estudiantes.....	10
10. Fuentes de Consulta.....	11
Conclusiones.....	13

Justificación

1. Escenario

Eres parte de coordinación académica. Necesitan un dashboard de reportes para identificar rendimiento, reprobación, asistencia y alumnos en riesgo. Tu app debe permitir filtrar por periodo y buscar alumnos, además de paginar resultados.

2. Stack

- Framework: Next.js (App Router).
- Base de Datos: PostgreSQL (utilizando pg para consultas directas).
- Validación: Zod para searchParams.
- Estilos: CSS
- Docker: Tanto web como la BD estan en contenedores, para que ambos se construyan mediante *docker compose up -d*

3. Estructura del Trabajo

- Front: Aplicación Next.js configurada para conectar con la base de datos mediante variables de entorno seguras (solo tiene acceso a views)
- db: Todo lo relacionado a SQL, con los siguientes archivos:
 - 01_schema.sql
 - 02_seed.sql
 - 03_indexes.sql
 - 04_reports_vw
 - 05_roles
- docker-compose.yml: Configuración de red interna y persistencia de datos mediante volúmenes
- .env: Gestión de credenciales y configuración de acceso al host de base de datos

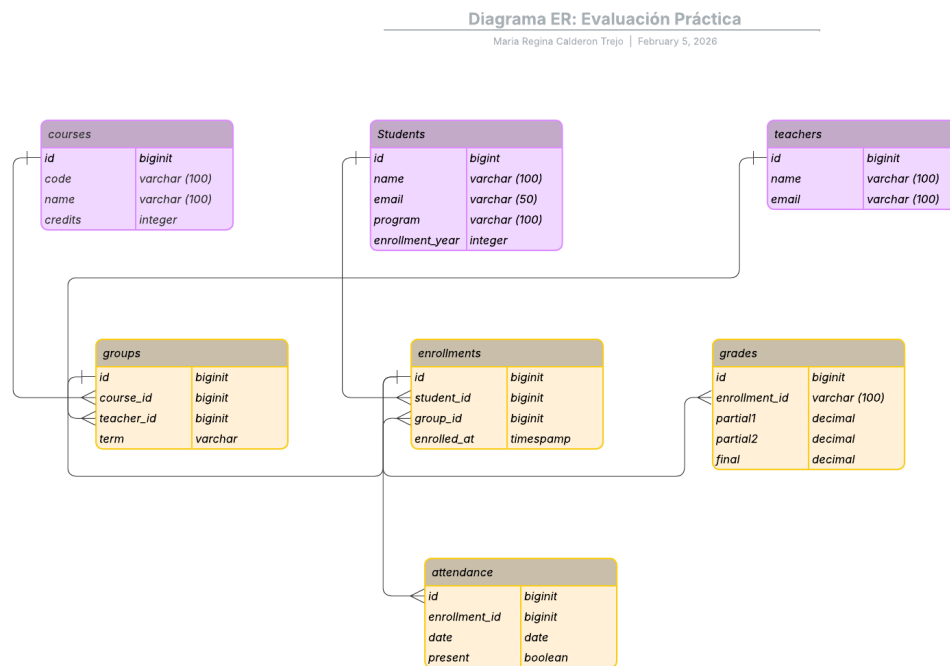
4. Flujo de inicialización

- 01_schema.sql: Esquema de la base de datos (Tablas y estructura general)
- 02_seed.sql: Insercción de datos semillas (datos de prueba)
- 03_indexes.sql: Optimización de velocidad sobre las tablas ya pobladas
- 04_reports_vw.sql: Transforman datos crudos en información estratégica
- 05_roles.sql: Creación del usuario web

5. BD

Schema.sql

La evaluación ya traía las entidades y sus respectivos atributos, solo fue la creación, en mi caso hice primero un diagrama, para después poder crear el script, se identificaron las tablas padre: students, teachers, courses.



Seed.sql

El Seed es el componente encargado de generar los datos necesarios para validar la arquitectura. Dado que el proyecto debe correr con *docker compose up --build*, este archivo garantiza que cualquier desarrollador o persona entre a un entorno de trabajo listo.

Indexes.sql

Se implementaron índices estratégicos en el archivo 03_indexes.sql para optimizar el rendimiento del motor de base de datos, se crearon índices en llaves foráneas para acelerar los JOINS entre tablas de gran tamaño.

Justificación de Indexes

- CREATE INDEX idx_groups_term ON groups(term): El campo term (periodo académico) es la clave de filtrado principal en casi todas las vistas del

sistema, sin este índice, PostgreSQL tendría que leer toda la tabla de grupos cada vez que un usuario cambia de periodo en el filtrado.

- `CREATE INDEX idx_students_search ON students(name, email)`: Este índice fue diseñado específicamente para la búsqueda de texto y la identificación de usuarios, ya que en reportes como `vw_students_at_risk`, es común que se quiera buscar a un alumno por su nombre, se incluyó el email, para cubrir las dos formas principales de identificar a los estudiantes.
- `CREATE INDEX idx_students_program ON students(program)`: La vista `vw_rank_students` utiliza la función de ventana `RANK() OVER`, este índice permite que el motor de la base de datos organice los grupos de estudiantes por carrera de mucho más rápido antes de calcular sus posiciones.

Reports_vw.sql

Las vistas se encuentran en `04_reports_vw.sql`. Next.js consume datos de vistas limpias, evitando que el código de la aplicación deba manejar lógica SQL compleja directamente, las vistas permiten mostrar solo los datos necesarios para los reportes sin exponer la estructura interna completa de las tablas base.

Justificacion de Vistas

- `vw_course_performance`: Permite detectar "materias filtro". Agrupa por programa y periodo, ayuda a identificar si un bajo rendimiento es generalizado o específico de una carrera en particular. Centraliza el cálculo del promedio aritmético de los tres (partial1, partial2, final). Utiliza una función de agregación condicional (`COUNT CASE`) para identificar la tasa de reprobados por materia.
- `vw_teacher_load`: Cruza la cantidad de grupos con el promedio general de calificaciones, para evaluar si una sobrecarga de alumnos impacta negativamente en la calidad educativa impartida por el docente. Se usa `HAVING COUNT(g.id) > 0` para garantizar excluir a docentes sin asignación activa.
- `vw_students_at_risk`: Define variables críticos (Promedio < 7 o Asistencia < 80%). Esta vista nos ayuda para una alerta temprana, permitiendo intervenciones antes de que el alumno abandone la carrera. Se utiliza una CTE (Common Table Expression) para aislar el cálculo antes de aplicar filtros

y se implementa NULLIF para prevenir errores de división por cero en el cálculo de asistencia.

- vw_attendance_by_group: Esta vista se puede considerar como un monitoreo, ya que al desglosar la asistencia por docente y curso, se puede identificar patrones de ausentismo. Usa LEFT JOIN para asegurar que las inasistencias no eliminen al grupo del reporte, se utiliza COALESCE para transformar los valores nulos a uno flotante (0.0), manteniendo la consistencia de los tipos de datos para el frontend.
- vw_rank_students: Reconocimiento de excelencia, ayuda a automatizar la identificación de los mejores promedios para programas de becas o méritos académicos. Implementa Funciones de Ventana (RANK() OVER) para calcular las posiciones, se implementó PARTITION BY que permite generar rankings independientes por carrera y periodo en una sola ejecución.

Roles.sql

En el archivo 05_rols.sql se aplicó el estándar de seguridad de Mínimo Privilegio, se creó el rol view_user específicamente para la capa web y se quitaron explícitamente los permisos de INSERT, UPDATE y DELETE, dejando solamente el permiso a los reportes.

6. Conexión de la BD al Frontend

La conexión inicia en PostgreSQL. En lugar de realizar consultas planas desde el frontend, se crearon Vistas Analíticas. Esto permite que Next.js ejecute un simple SELECT * FROM vw_nombre en lugar de lidiar con queries más complejas, dejando toda la lógica de negocio en la BD. Para conectar la base de datos, se utilizó el driver pg dentro de Server Components, al realizar la conexión exclusivamente en el servidor, las credenciales de la base de datos y la cadena de conexión nunca se exponen al navegador del usuario.

En lugar de usar APIs externas, utilice Server Components, dentro de los archivos page.tsx, se importó la función de conexión a la base de datos (query). Al ser componentes de servidor, la consulta SQL se ejecuta directamente en Node.js mientras Next.js construye la página.

Se implementaron los componentes de filtro y paginación, el filtrado manipula los searchParams de la URL, para que al utilizar o cambiar un filtro, Next.js re-ejecuta

la consulta SQL agregando cláusulas WHERE dinámicas, asegurando así que solo se descarguen los datos necesarios. Para la paginación, se usó LIMIT y OFFSET en las consultas SQL. Esto permite navegar por grandes volúmenes de datos en bloques (10 por página).

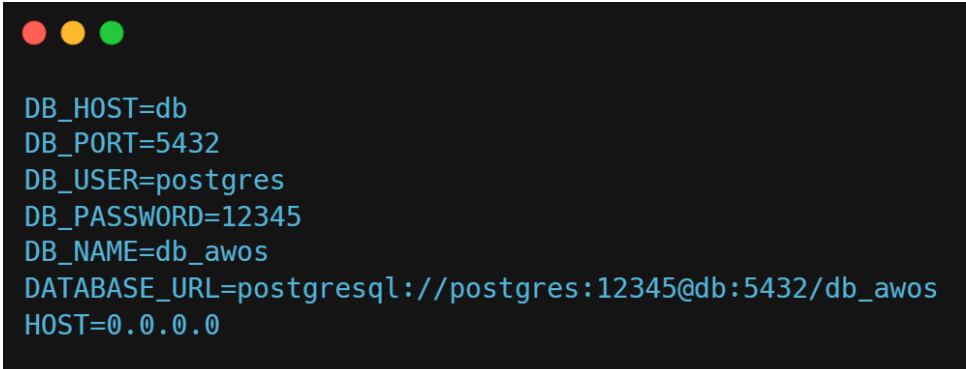
7. Docker-compose

Docker Compose empaqueta tanto la aplicación web como la base de datos con sus configuraciones específicas, permitiendo que cualquier persona pueda levantar el sistema completo con un solo comando (*docker compose up -d*), asegurando que las versiones de Node.js y Postgres sean exactamente las mismas en todos los despliegues.

8. Variables de entorno

.env

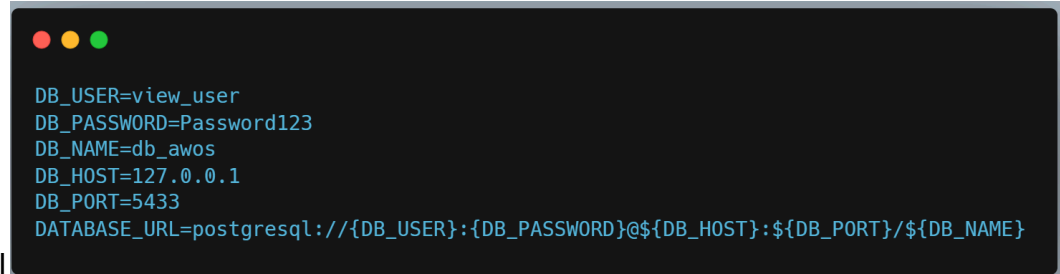
En la raíz crea un archivo .env con los siguientes datos



```
DB_HOST=db
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=12345
DB_NAME=db_awos
DATABASE_URL=postgresql://postgres:12345@db:5432/db_awos
HOST=0.0.0.0
```

.env.local

En la raíz de tu carpeta Front crea el archivo .env.local con los siguientes datos

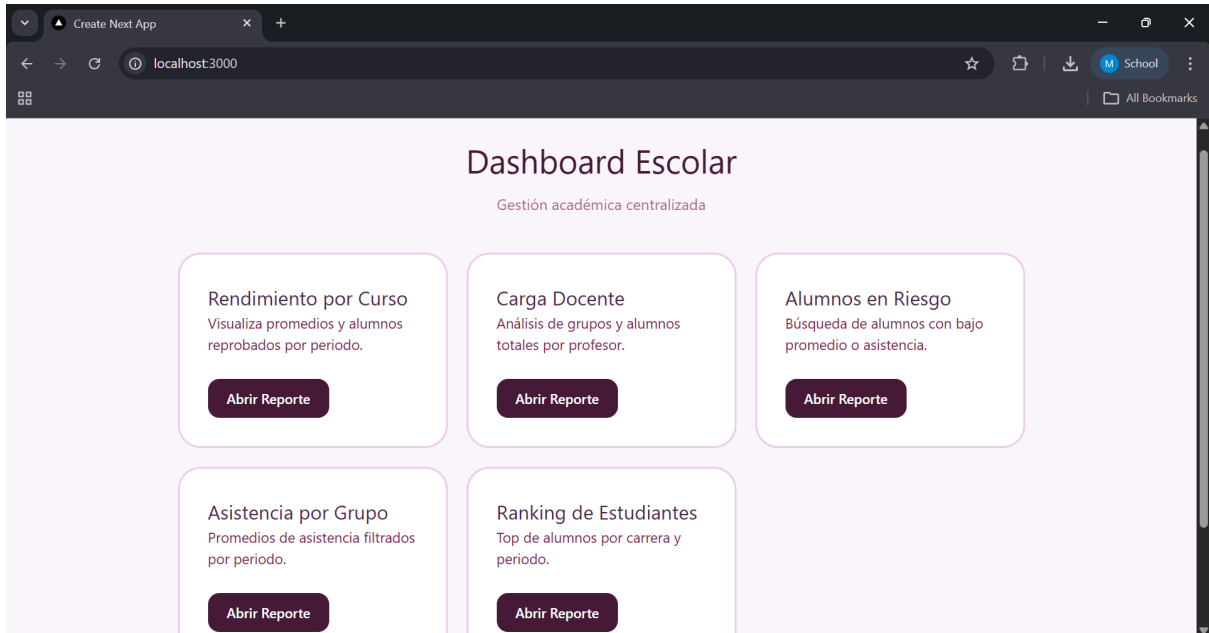


```
DB_USER=view_user
DB_PASSWORD=Password123
DB_NAME=db_awos
DB_HOST=127.0.0.1
DB_PORT=5433
DATABASE_URL=postgresql://${DB_USER}:${DB_PASSWORD}@${DB_HOST}:${DB_PORT}/${DB_NAME}
```

9. Interfaz

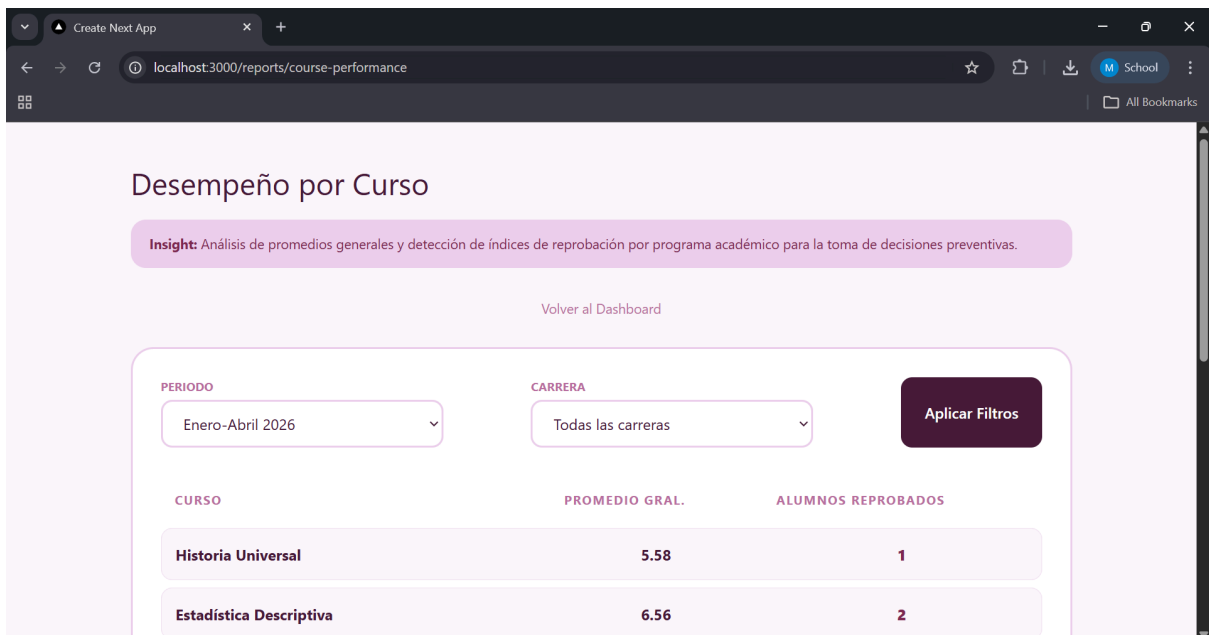
- Dashboard

Muestra los 5 reportes, cada uno en su respectiva card, con un título, una breve descripción y el link hacia la vista.



- Rendimiento por curso

Trabaja con la vista `vw_course_performance`



- Carga Docente

Trabaja con la vista vw_teacher_load

The screenshot shows a web browser window with the URL `localhost:3000/reports/teacher-load`. The page title is "Carga Académica por Docente". Below the title is an insight box: "Insight: Este reporte permite equilibrar la distribución de alumnos y grupos, asegurando la calidad educativa." A link "Volver al Dashboard" is visible. Below this is a search bar with the placeholder "Buscar docente..." and a "Buscar" button. The main content is a table with the following data:

DOCENTE	PERIODO	GRUPOS	TOTAL ALUMNOS	PROMEDIO
Dr. Gerardo Nuñez	Enero-Abril 2026	1	6	6.73
Dr. Iván Paredes	Enero-Abril 2026	1	6	7.38

- Alumnos en Riesgo

Trabaja con la vista vw_students_at_risk

The screenshot shows a web browser window with the URL `localhost:3000/reports/students-at-risk`. The page title is "Alumnos en Riesgo Académico". Below the title is an insight box: "Insight: Este reporte identifica estudiantes vulnerables por faltas o promedio menor a 80, facilitando la intervención de los docentes." A link "Volver al Dashboard" is visible. Below this is a search bar with the placeholder "Nombre, carrera o email..." and a "Buscar" button. The main content is a table with the following data:

CARRERA	NOMBRE	CORREO	PROMEDIO	ASISTENCIA
Ciencia de Datos	Ximena Lara	x.lara@correo.com	6.23	50.00%
Ciencia de Datos	Sara Villa	s.villa@correo.com	6.36	33.33%

- Asistencia por Grupo

Trabaja con la vista vw_attendance_by_group

The screenshot shows a web browser at localhost:3000/reports/attendance-group. The page title is 'Asistencia por Grupo'. An insight box states: 'Este reporte detecta grupos con baja asistencia para implementar estrategias de motivación y mejorar el compromiso estudiantil.' A link 'Volver al Dashboard' is present. The report includes a filter for 'PERIODO ACADÉMICO' set to 'Enero-Abril 2025' and a 'Filtrar' button. The data is presented in a table with columns: ID, CURSO / MATERIA, PROFESOR, SESIONES, and ASISTENCIA.

ID	CURSO / MATERIA	PROFESOR	SESIONES	ASISTENCIA
#1	Matemáticas Avanzadas	Dr. Roberto García	1	0.00%
#1	Matemáticas Avanzadas	Dr. Roberto García	2	50.00%

- Ranking de Estudiantes

Trabaja con la vista vw_rank_students

The screenshot shows a web browser at localhost:3000/reports/student-rank. The page title is 'Ranking de Estudiantes'. An insight box states: 'Este reporte visualiza el desempeño sobresaliente para la asignación de becas y reconocimientos.' A link 'Volver al Dashboard' is present. The report features a highlighted section for 'PRIMER LUGAR DE EXCELENCIA — INGENIERÍA EN SISTEMAS' with the student 'Miguel Jara (9.10)'. Below this, there are filters for 'Programa' (set to 'Ingeniería en Sistemas') and 'Periodo' (set to 'Enero-Abril 2025'), along with a 'Filtrar' button. The data is presented in a table with columns: POSICIÓN, NOMBRE DEL ESTUDIANTE, and PROMEDIO FINAL.

POSICIÓN	NOMBRE DEL ESTUDIANTE	PROMEDIO FINAL
----------	-----------------------	----------------

10. Fuentes de Consulta

Le pedí a Gemini que me generará mi archivo seed.sql y que me ayudará con un error que me salió por el navegador que estaba usando.

Error:

```
## Error Type
Console Error

## Error Message
A tree hydrated but some attributes of the server rendered HTML didn't match the client properties.
This won't be patched up. This can happen if a SSR-ed Client Component used:

- A server/client branch `if (typeof window !== 'undefined')`.
- Variable input such as `Date.now()` or `Math.random()` which changes each time it's called.
- Date formatting in a user's locale which doesn't match the server.
- External changing data without sending a snapshot of it along with the HTML.
- Invalid HTML tag nesting.

It can also happen if the client has a browser extension installed which messes with the HTML before
React loaded.

https://react.dev/link/hydration-mismatch

...
  <RenderFromTemplateContext>
    <ScrollAndFocusHandler segmentPath={[[...]]}>
      <InnerScrollAndFocusHandler segmentPath={[[...]]} focusAndScrollRef={{apply:false, ...}}>
        <ErrorBoundary errorComponent={undefined} errorStyles={undefined} errorScripts={undefined}>
          <LoadingBoundary name="student-rank/" loading={null}>
            <HTTPAccessFallbackBoundary notFound={undefined} forbidden={undefined} unauthorized=
{undefined}>
              <RedirectBoundary>
                <RedirectErrorBoundary router={{...}}>
                  <InnerLayoutRouter url={"/reports..."} tree={[[...]]} params={{}} cacheNode={{rsc:
<Fragment>, ...}} ...>
                    <SegmentViewNode type="page" pagePath="/OneDrive/...">
                      <SegmentTrieNode>
                        <RankingPage>
                          <main>
                            <h1>
                              <p>
                                <form method="GET">
                                  <select
                                    name="program"
                                    defaultValue="Historia Universal"
                                    fdprocessedid="lywfp"
                                  >
                                    <input
                                      type="text"
                                      name="term"
                                      defaultValue="2025-A"
                                      fdprocessedid="mms0hj"
                                    >
                                      <button
                                        type="submit"
                                        fdprocessedid="wzowwe"
                                      >
                                        Filtrar
                                  ...
                                ...
                              ...
                            ...
                          ...
                        ...
                      ...
                    ...
                  ...
                ...
              ...
            ...
          ...
        ...
      ...
    ...
  ...

at select (<anonymous>:null:null)
at RankingPage (src/app/reports/student-rank/page.tsx:51:9)

## Code Frame
49 |
50 |     <form method="GET">
> 51 |       <select name="program" defaultValue={program}>
    |       ^
52 |       {VALID_PROGRAMS.map(p => <option key={p} value={p}>{p}</option>)}
53 |     </select>
54 |     <input type="text" name="term" defaultValue={term} />

Next.js version: 16.1.6 (Turbopack)
```

Respuesta:

Este error es común en navegadores como **Chrome o Edge** y se debe a una característica del navegador llamada "Auto-completado" o extensiones de gestión de formularios.

¿Por qué sucede?

El navegador inyecta automáticamente el atributo `fdprocessedid` en tus elementos `<select>`, `<input>` y `<button>` en cuanto la página carga para gestionar el procesamiento del formulario. Como el **Servidor** (Next.js) no envió ese atributo, pero el **Cliente** (Navegador) lo agregó apenas recibió el HTML, React detecta una diferencia y lanza el error de "Hydration Mismatch".

La Solución

Para resolver esto en un entorno de desarrollo con Next.js, debes usar la propiedad `suppressHydrationWarning`. Esto le indica a React que ignore las pequeñas discrepancias de atributos causadas por extensiones o el motor del navegador.

Conclusiones

Este proyecto me permitió entender cómo hacer que la base de datos haga el trabajo pesado, en lugar de complicar la página web con cálculos difíciles, dejé que la base de datos se encargará de la lógica detrás (todos los cálculos), esto hace que la página cargue rápido.

Se configuró el sistema para que la página web solo pueda leer los reportes, pero no borrar ni cambiar nada de la información original.

Se implementaron filtros para volver más fácil la búsqueda de algún campo en específico, y para reducir la carga visual se implementó la paginación, en mi caso reduje a 10 columnas por página.

Gracias a Docker, con un solo comando se instala todo: se crean las tablas, se llenan los datos de prueba y se pone en marcha la creación de las vistas web.

En resumen, se creó una app web que es robusta y ayuda al escenario puesto de forma rápida y segura.